

Name: - Shubham Das
Date:- 31/08/2026

S. A. P I. D: - 590033500
Course: - B.TECH (CSE)

Question 1: - Write a C program to find the time period of a simple pendulum when length and gravity are given.

Part 1: Thorough understanding of the problem statement

Input: - L: - Length of the pendulum
g: - Acceleration due to gravity (in meters per second squared)

Output: - T: - Time period of the pendulum (in seconds)

Formula: - $2 * \pi * \sqrt{L / g}$

Part 2: - Writing, compiling and evaluating the algorithm

Step 1: - Start

Step 2: - Declare variables 'L', 'g', 'T' as double-precision floating-point data types (double).

Step 3: - Declare constant PI as 3.14159265358979323846.

Step 4: - User prompt: - "Enter the length of the pendulum (L) in meters: "

Step 5: - Read value into the variable 'L'.

Step 6: - User prompt: - "Enter the acceleration due to gravity (g) in m/s^2: "

Step 7: - Read value in variable 'g'.

Step 8: - Input Validation Check: -

If $L \leq 0$ OR $g \leq 0$ then:

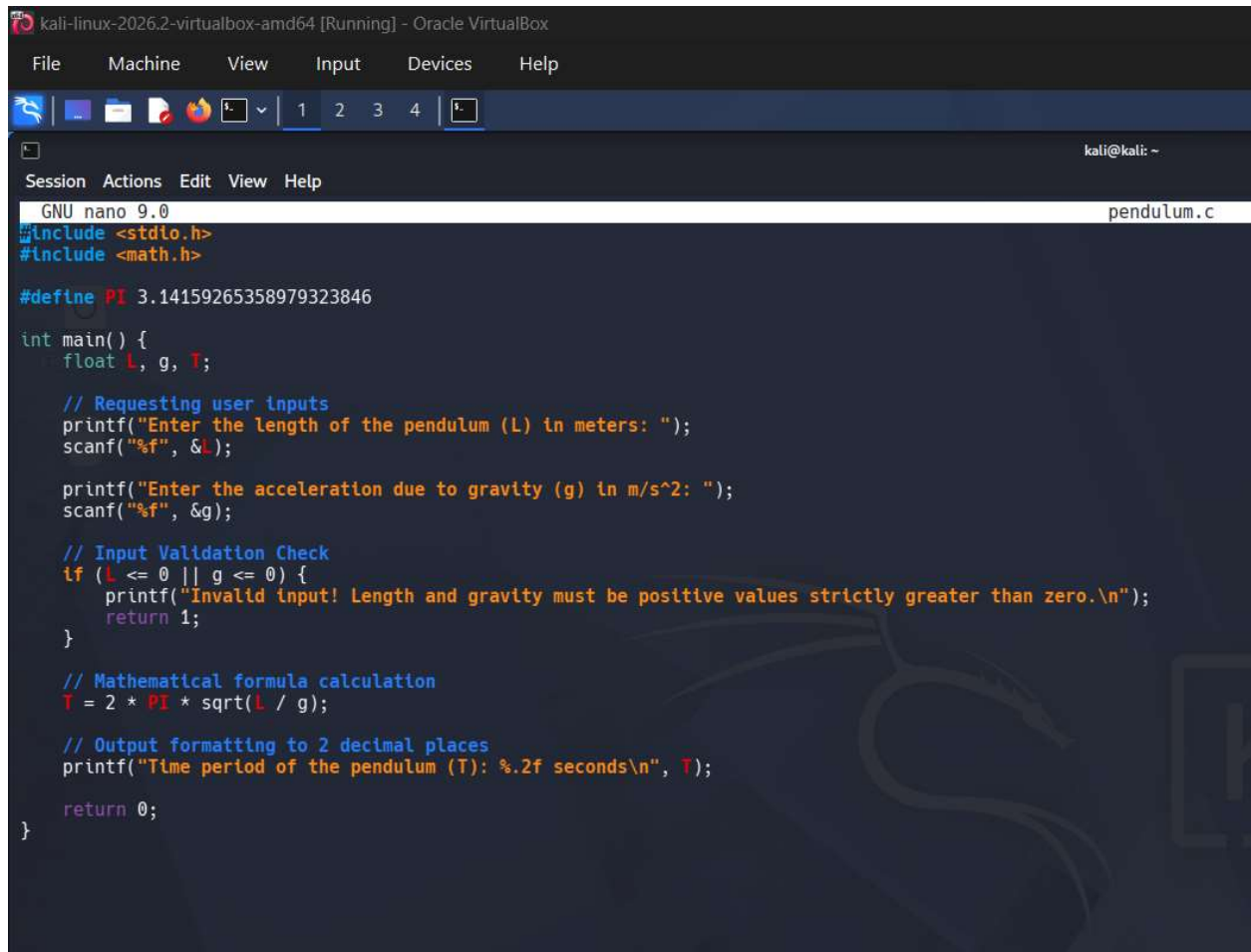
Print error message: "Invalid input! Length and gravity must be positive values strictly greater than zero."

Terminate programme with error state ('exit code 1')

Step 9: - Display formatted result: " Time Period (T) = %.2f seconds", specifying 'T'.

Step 10: - Stop

Part 3: - Writing and compiling the C programme



The screenshot shows a Kali Linux virtual machine running in Oracle VM VirtualBox. The terminal window is titled 'kali-linux-2026.2-virtualbox-amd64 [Running] - Oracle VirtualBox'. The terminal interface includes a menu bar with 'File', 'Machine', 'View', 'Input', 'Devices', and 'Help'. Below the menu bar is a toolbar with icons for file operations and a tab bar showing tabs 1, 2, 3, 4, and 5. The terminal prompt is 'kali@kali: ~'. The user has opened a file named 'pendulum.c' in the nano text editor. The code in the file is a C program for calculating the time period of a pendulum. It includes headers for stdio.h and math.h, defines a constant PI, and contains a main function that prompts the user for length (L) and gravity (g), validates the inputs, calculates the time period (T) using the formula $T = 2 * \pi * \sqrt{L / g}$, and prints the result formatted to two decimal places.

```
GNU nano 9.0 pendulum.c
#include <stdio.h>
#include <math.h>

#define PI 3.14159265358979323846

int main() {
    float L, g, T;

    // Requesting user inputs
    printf("Enter the length of the pendulum (L) in meters: ");
    scanf("%f", &L);

    printf("Enter the acceleration due to gravity (g) in m/s^2: ");
    scanf("%f", &g);

    // Input Validation Check
    if (L <= 0 || g <= 0) {
        printf("Invalid input! Length and gravity must be positive values strictly greater than zero.\n");
        return 1;
    }

    // Mathematical formula calculation
    T = 2 * PI * sqrt(L / g);

    // Output formatting to 2 decimal places
    printf("Time period of the pendulum (T): %.2f seconds\n", T);

    return 0;
}
```

Part 4: - Running and executing the algorithm on a platform (Kali Linux VM)

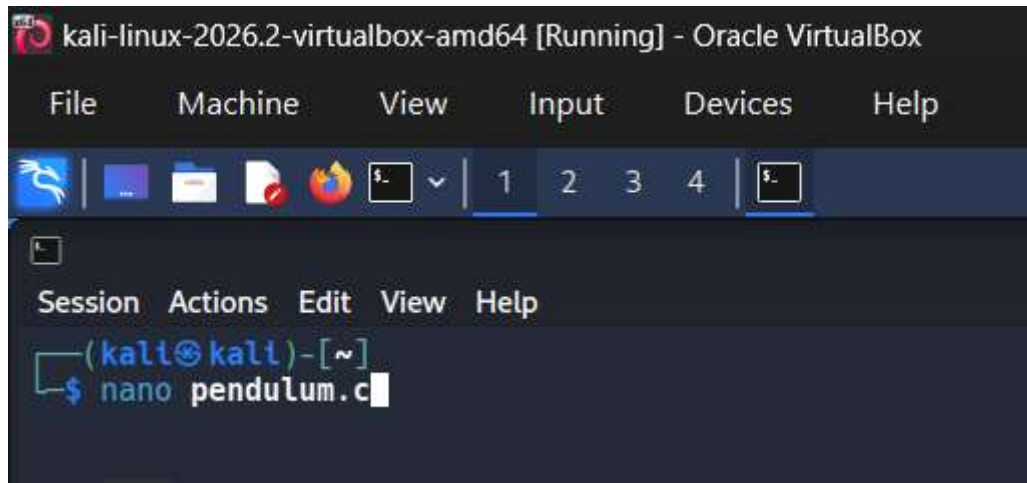
Step 1: - Open the Terminal

Step 2: - Create the Source File

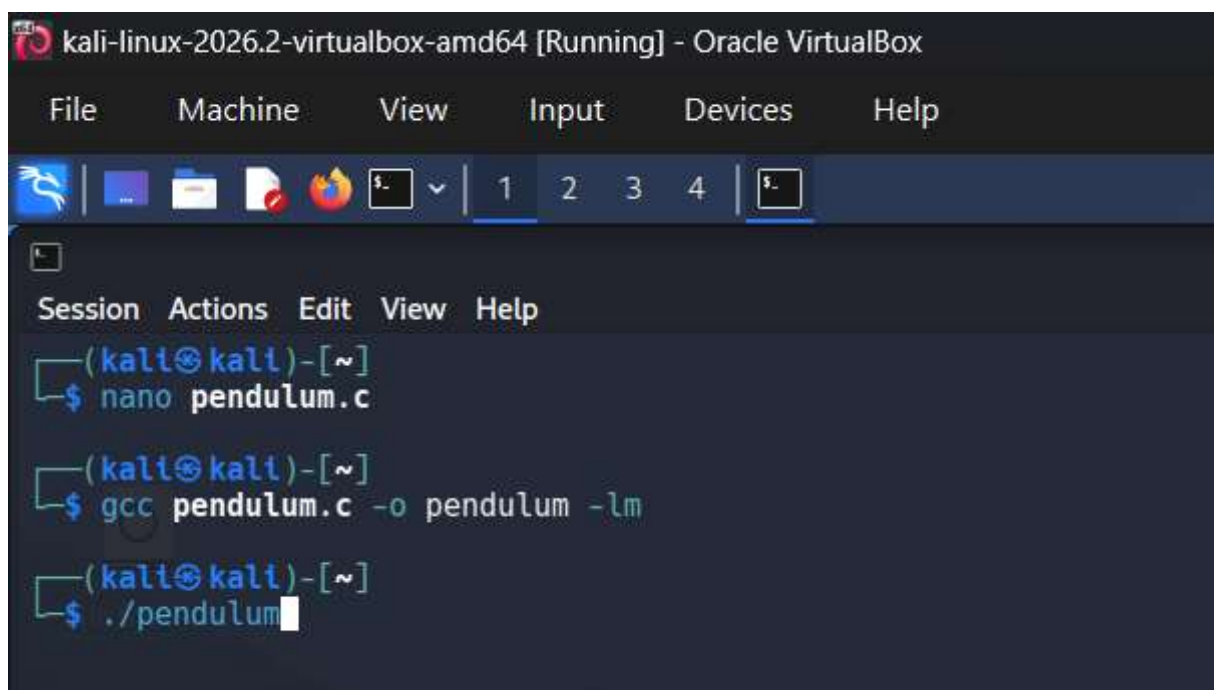
Step 3: - Write and Save Code

Step 4: - Compile the programme.

Step 5: - Run the Binary Executable programme



```
kali-linux-2026.2-virtualbox-amd64 [Running] - Oracle VirtualBox
File Machine View Input Devices Help
(kali@kali)-[~]
$ nano pendulum.c
```



```
kali-linux-2026.2-virtualbox-amd64 [Running] - Oracle VirtualBox
File Machine View Input Devices Help
(kali@kali)-[~]
$ nano pendulum.c
(kali@kali)-[~]
$ gcc pendulum.c -o pendulum -lm
(kali@kali)-[~]
$ ./pendulum
```

```
kali-linux-2026.2-virtualbox-amd64 [Running] - Oracle VirtualBox
File Machine View Input Devices Help
1 2 3 4

kali@kali: ~
Session Actions Edit View Help
(kali@kali)-[~]
$ nano pendulum.c
(kali@kali)-[~]
$ gcc pendulum.c -o pendulum -lm
(kali@kali)-[~]
$ ./pendulum
Enter the length of the pendulum (L) in meters: 1.0
Enter the acceleration due to gravity (g) in m/s^2: 9.8
Time period of the pendulum (T): 2.01 seconds
(kali@kali)-[~]
$ ./pendulum
Enter the length of the pendulum (L) in meters: 2.5
Enter the acceleration due to gravity (g) in m/s^2: 9.81
Time period of the pendulum (T): 3.17 seconds
(kali@kali)-[~]
$ ./pendulum
Enter the length of the pendulum (L) in meters: 0.5
Enter the acceleration due to gravity (g) in m/s^2: 9.8
Time period of the pendulum (T): 1.42 seconds
(kali@kali)-[~]
$
```

```
kali-linux-2026.2-virtualbox-amd64 [Running] - Oracle VirtualBox
File Machine View Input Devices Help

(kali@kali)-[~]
$ nano pendulum.c

(kali@kali)-[~]
$ gcc pendulum.c -o pendulum -lm

(kali@kali)-[~]
$ ./pendulum
Enter the length of the pendulum (L) in meters: 1.0
Enter the acceleration due to gravity (g) in m/s^2: 9.8
Time period of the pendulum (T): 2.01 seconds

(kali@kali)-[~]
$ ./pendulum
Enter the length of the pendulum (L) in meters: 2.5
Enter the acceleration due to gravity (g) in m/s^2: 9.81
Time period of the pendulum (T): 3.17 seconds

(kali@kali)-[~]
$ ./pendulum
Enter the length of the pendulum (L) in meters: 0.5
Enter the acceleration due to gravity (g) in m/s^2: 9.8
Time period of the pendulum (T): 1.42 seconds

(kali@kali)-[~]
$ ./pendulum
Enter the length of the pendulum (L) in meters: -1
Enter the acceleration due to gravity (g) in m/s^2: 7
Invalid input! Length and gravity must be positive values strictly greater than zero.

(kali@kali)-[~]
$
```

Part 5: - Compiling, executing and evaluating against all the test cases

Upon compiling and executing the code with the inputs from Part 4, we will check whether or not the calculated outputs match the expected outputs.

Test Case 1 (1.0, 9.8): Evaluates to $2 * \text{PI} * \sqrt{1.0 / 9.8} = 2.01$. Actual output is 2.01 seconds. **(Pass)**

Test Case 2 (2.5, 9.81): Evaluates to $2 * \text{PI} * \sqrt{2.5 / 9.81} = 3.17$. Actual output is 3.17 seconds. **(Pass)**

Test Case 3 (0.5, 9.8): Evaluates to $2 * \text{PI} * \sqrt{0.5 / 9.8} = 1.42$. Actual output is 1.42 seconds. **(Pass)**

TEST CASE	INPUT(S)	EXPECTED OUTPUT(S)	ACTUAL OUTPUT(S)	RESULT (PASS/FAIL)
1	L = 1.0 meters	2.01 seconds	2.01 seconds	PASS

	g = 9.8 m/s²			
2	L = 2.5 meters g = 9.81 m/s²	3.17 seconds	3.17 seconds	PASS
3	L = 0.5 meters g = 98 m/s²	1.42 seconds	1.42 seconds	PASS

Part 6: - Evaluation

1. The algorithm executes a strict sequence of standard mathematical operations (division, square root calculation, and scalar multiplication).
 2. If $g = 0$, direct computation yields infinity (∞). The condition check ' $g \leq 0$ ' prevents undefined execution.
-

Question 2: - Write a C program to find stress force and area are given.

Part 1: Thorough understanding of the problem statement

Input: - 'force' (Variable type: 'float', Units: Newtons or N)
 'area' (Variable type: 'float', Units: Square meters or m²)

Output: - 'stress' (Variable type: 'float', Unit: Pascals or N/m²)

Formula: - 'stress = force / area' ;

Part 2: - Writing, compiling and evaluating the algorithm

Step 1: - Start

Step 2: - Declare floating-point variables 'force', 'area', and 'stress'.

Step 3: - Prompt the user to enter the value for 'force'.

Step 4: - Read the input into the 'force' variable.

Step 5: - Prompt the user to enter the value for 'area'.

Step 6: - Read the input into the 'area' variable.

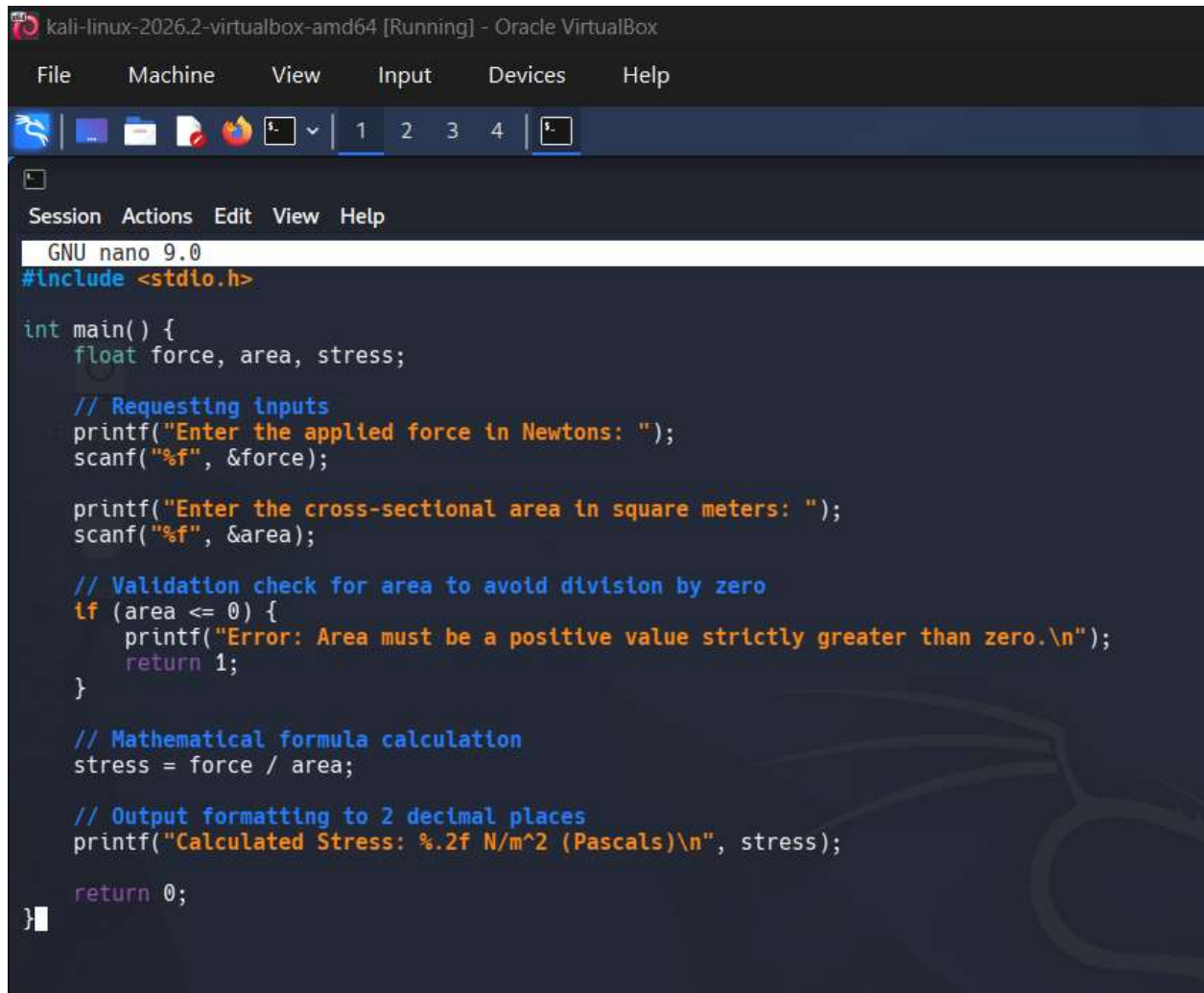
Step 7: - Check if 'area' is equal to '0'. If true, display a "Division by zero" error message and terminate the programme to prevent crashing.

Step 8: - Calculate stress using the formula 'stress = force / area'.

Step 9: - Display the calculated 'stress' value.

Step 10: - Stop

Part 3: - Writing and compiling the C programme



```
GNU nano 9.0
#include <stdio.h>

int main() {
    float force, area, stress;

    // Requesting inputs
    printf("Enter the applied force in Newtons: ");
    scanf("%f", &force);

    printf("Enter the cross-sectional area in square meters: ");
    scanf("%f", &area);

    // Validation check for area to avoid division by zero
    if (area <= 0) {
        printf("Error: Area must be a positive value strictly greater than zero.\n");
        return 1;
    }

    // Mathematical formula calculation
    stress = force / area;

    // Output formatting to 2 decimal places
    printf("Calculated Stress: %.2f N/m^2 (Pascals)\n", stress);

    return 0;
}
```

Part 4: - Running and executing the algorithm on a platform (Kali Linux VM)

Step 1: - Open the Terminal

Step 2: - Create the Source File

Step 3: - Write and Save Code

Step 4: - Compile the programme.

Step 5: - Run the Binary Executable programme

```
kali-linux-2026.2-virtualbox-amd64 [Running] - Oracle VirtualBox
File Machine View Input Devices Help
[Icons] | 1 2 3 4 | [Terminal Icon]

Session Actions Edit View Help
(kali@kali)-[~]
$ nano stress.c

(kali@kali)-[~]
$ gcc stress.c -o stress
```

```
kali-linux-2026.2-virtualbox-amd64 [Running] - Oracle VirtualBox
File Machine View Input Devices Help
[Icons] | 1 2 3 4 | [Terminal Icon]

Session Actions Edit View Help
(kali@kali)-[~]
$ ./stress
Enter the applied force in Newtons: 100
Enter the cross-sectional area in square meters: 2
Calculated Stress: 50.00 N/m^2 (Pascals)

(kali@kali)-[~]
$ ./stress
Enter the applied force in Newtons: 45.5
Enter the cross-sectional area in square meters: 5
Calculated Stress: 9.10 N/m^2 (Pascals)

(kali@kali)-[~]
$ ./stress
Enter the applied force in Newtons: 150
Enter the cross-sectional area in square meters: 0
Error: Area must be a positive value strictly greater than zero.

(kali@kali)-[~]
$
```

Part 5: - Compiling, executing and evaluating against all the test cases

Upon compiling and executing the code with the inputs from Part 4, we will check whether or not the calculated outputs match the expected outputs.

Test Case 1 (100, 2): Evaluates to $100 / 2 = 50.00$. Actual Output is 50.00 N/m² (Pascals). (Pass)

Test Case 2 (45.5, 5): Evaluates to $45.5 / 5 = 9.10$. Actual Output is 9.10 N/m² (Pascals). (Pass)

Test Case 3 (150, 0): Evaluates to $150 / 0 = \text{'ERROR'}$. Division by zero is undefined. (Pass)

TEST CASE	INPUT(S)	EXPECTED OUTPUT(S)	ACTUAL OUTPUT(S)	RESULT (PASS/FAIL)
1	force = 100 area = 2	50.00 N/m ² (Pascals)	50.00 N/m ² (Pascals)	PASS
2	force = 45.5 area = 5	9.10 N/m ² (Pascals)	9.10 N/m ² (Pascals)	PASS
3	force = 150 area = 0	ERROR	ERROR	PASS

Part 6: - Evaluation

The algorithm executes in constant time and requires constant auxiliary space, as it relies entirely on direct arithmetic operations and three scalar variables. This check prevents undefined behavior and system crashes caused by divide-by-zero errors or physically impossible negative areas, ensuring safe and predictable execution under all input conditions.

Question 3: - Write a C programme to find the transformer secondary current when primary current, primary voltage, and secondary voltage are given.

Part 1: Thorough understanding of the problem statement

Input: - 'Vp' (Primary Voltage, Variable type: 'float', Unit: Volts or 'V')
 'Ip' (Primary Current, Variable type: 'float', Units: Amperes or 'A')
 'Vs' (Secondary Voltage, Variable type: 'float', Units: Volts or 'V')

Output: - 'Is' (Secondary Current, Variable type: 'float', Unit: Amperes or A)

Formula: - $Is = (Vp * Ip) / Vs$;

Part 2: - Writing, compiling and evaluating the algorithm

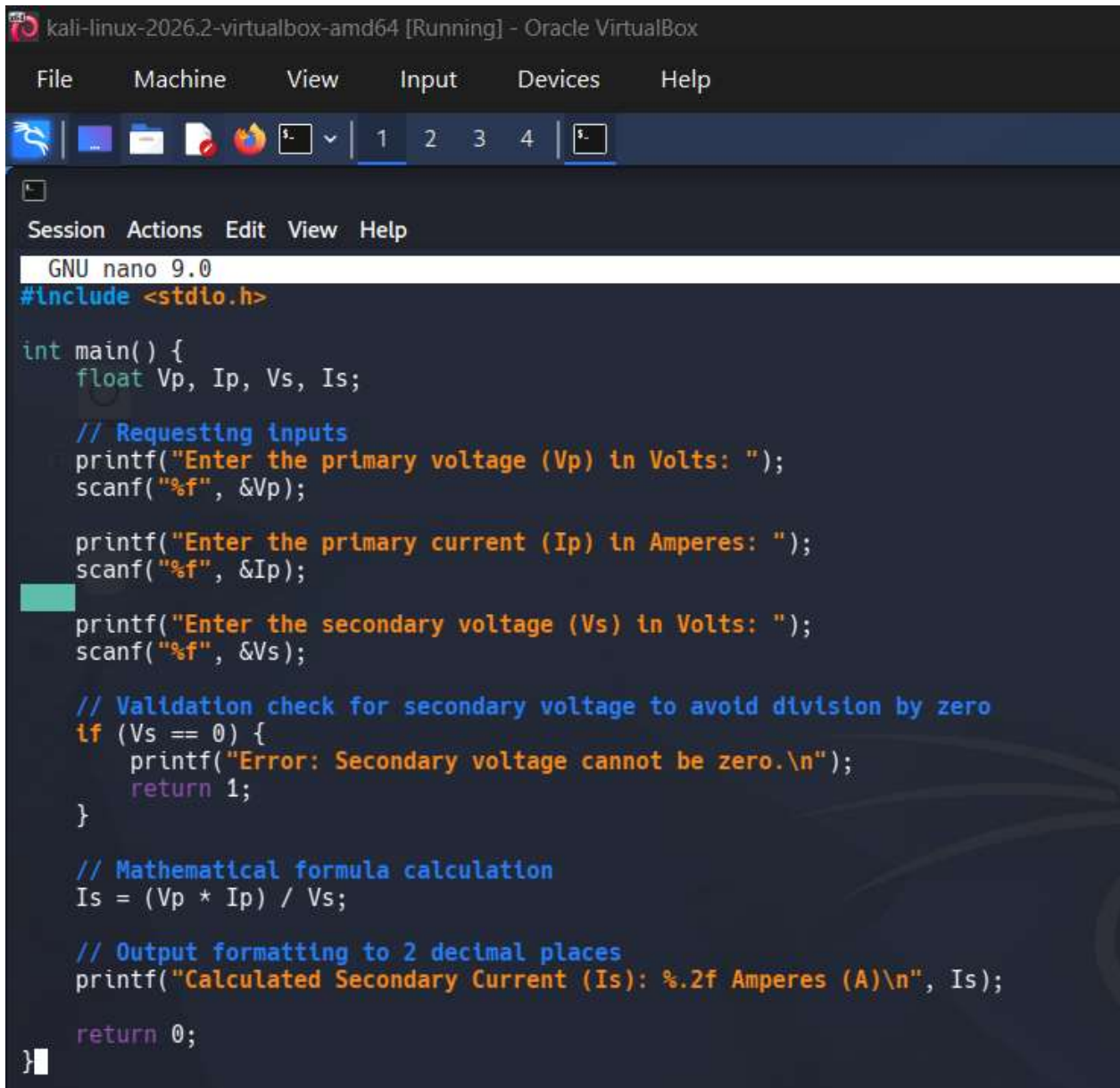
Step 1: - Start

Step 2: - Declare floating-point variables 'Vp', 'Ip', 'Vs', and 'Is'.

Step 3: - Prompt the user to enter the primary voltage ('Vp').

- Step 4:** - Read the input into the 'Vp' variable.
- Step 5:** - Prompt the user to enter the primary current ('Ip').
- Step 6:** - Read the input into the 'Ip' variable.
- Step 7:** - Prompt the user to enter the secondary voltage ('Vs').
- Step 8:** - Read the input into the 'Vs' variable.
- Step 9:** - Check if 'Vs' is equal to '0'. If true, display a division-by-zero error message and terminate the programme.
- Step 10:** - Calculate the secondary current using the formula $I_s = (V_p * I_p) / V_s$.
- Step 11:** - Display the calculated 'Is' value.
- Step 12:** - Stop

Part 3: - Writing and compiling the C programme



```
GNU nano 9.0
#include <stdio.h>

int main() {
    float Vp, Ip, Vs, Is;

    // Requesting inputs
    printf("Enter the primary voltage (Vp) in Volts: ");
    scanf("%f", &Vp);

    printf("Enter the primary current (Ip) in Amperes: ");
    scanf("%f", &Ip);

    printf("Enter the secondary voltage (Vs) in Volts: ");
    scanf("%f", &Vs);

    // Validation check for secondary voltage to avoid division by zero
    if (Vs == 0) {
        printf("Error: Secondary voltage cannot be zero.\n");
        return 1;
    }

    // Mathematical formula calculation
    Is = (Vp * Ip) / Vs;

    // Output formatting to 2 decimal places
    printf("Calculated Secondary Current (Is): %.2f Amperes (A)\n", Is);

    return 0;
}
```

Part 4: - Running and executing the algorithm on a platform (Kali Linux VM)

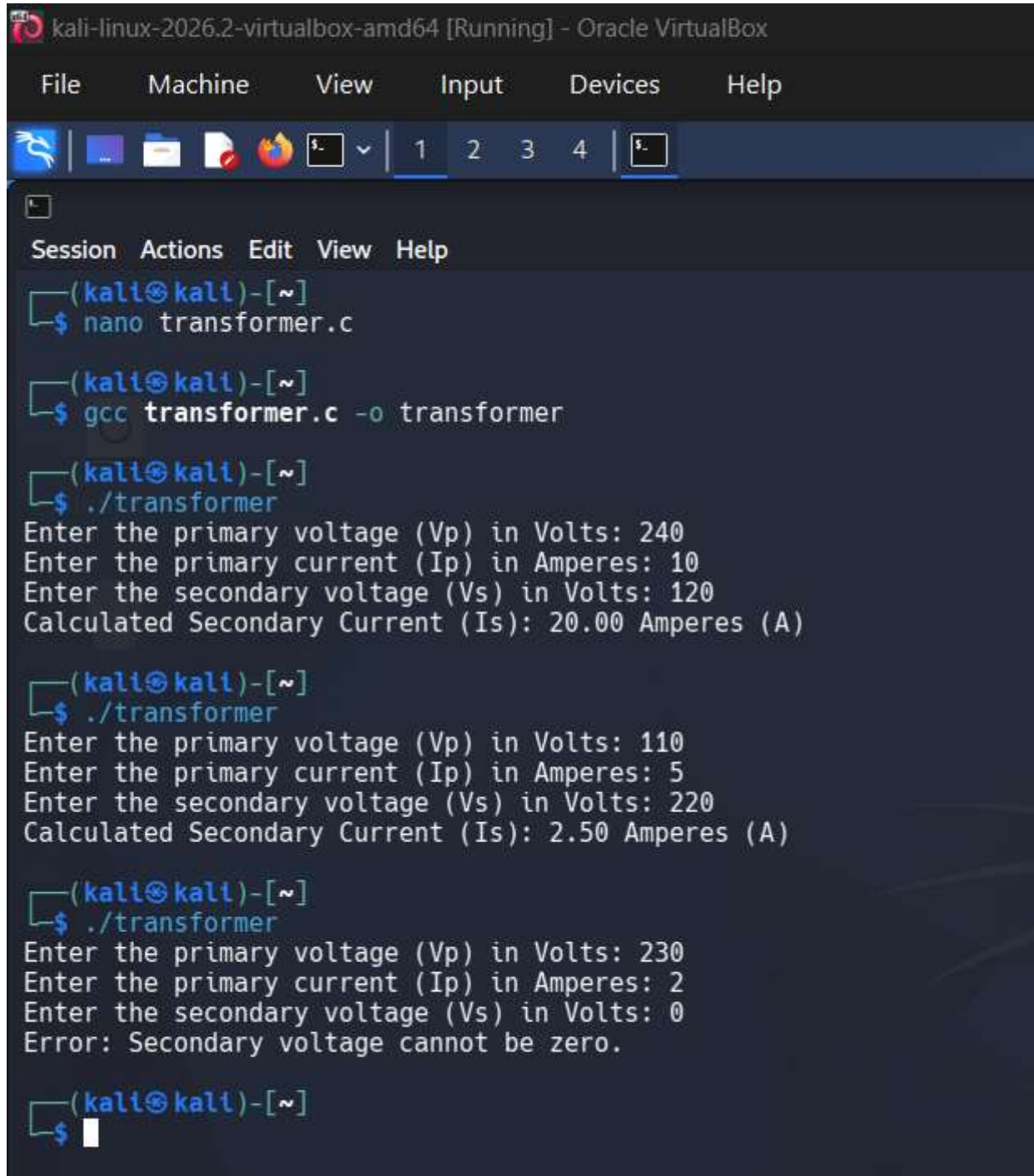
Step 1: - Open the Terminal

Step 2: - Create the Source File

Step 3: - Write and Save Code

Step 4: - Compile the programme.

Step 5: - Run the Binary Executable programme



```
kali-linux-2026.2-virtualbox-amd64 [Running] - Oracle VirtualBox
File Machine View Input Devices Help

(kali@kali)-[~]
$ nano transformer.c

(kali@kali)-[~]
$ gcc transformer.c -o transformer

(kali@kali)-[~]
$ ./transformer
Enter the primary voltage (Vp) in Volts: 240
Enter the primary current (Ip) in Amperes: 10
Enter the secondary voltage (Vs) in Volts: 120
Calculated Secondary Current (Is): 20.00 Amperes (A)

(kali@kali)-[~]
$ ./transformer
Enter the primary voltage (Vp) in Volts: 110
Enter the primary current (Ip) in Amperes: 5
Enter the secondary voltage (Vs) in Volts: 220
Calculated Secondary Current (Is): 2.50 Amperes (A)

(kali@kali)-[~]
$ ./transformer
Enter the primary voltage (Vp) in Volts: 230
Enter the primary current (Ip) in Amperes: 2
Enter the secondary voltage (Vs) in Volts: 0
Error: Secondary voltage cannot be zero.

(kali@kali)-[~]
$
```

Part 5: - Compiling, executing and evaluating against all the test cases

Upon compiling and executing the code with the inputs from Part 4, we will check whether or not the calculated outputs match the expected outputs.

Test Case 1 (240, 10, 120): Evaluating to $I_s = (V_p * I_p) / V_s$; $(240 * 10) / 120 = 20.00$. Actual output is 20. 00 Amperes (A). **(Pass)**

Test Case 2 (110, 5, 220): Evaluating to $I_s = (V_p * I_p) / V_s$; $(110 * 5) / 220 = 2.50$. Actual output is 2. 50 Amperes (A). **(Pass)**

Test Case 3 (230, 2, 0): Evaluating to $I_s = (V_p * I_p) / V_s$; $(230 * 2) / 0 = \text{ERROR}$. The actual output is 'ERROR'. **(Pass)**

TEST CASE	INPUT ('Vp', 'Ip', 'Vs')	EXPECTED OUTPUT ('Is')	ACTUAL OUTPUT ('Is')	RESULT (PASS/FAIL)
1	Vp = 240 Ip = 10 Vs = 120	20.00 Amperes	20.00 Amperes	PASS
2	Vp = 110 Ip = 5 Vs = 220	2.50 Amperes	2.50 Amperes	PASS
3	Vp = 230 Ip = 2 Vs = 0	Error: Cannot be zero	Error: Cannot be zero	PASS

Part 6: - Evaluation

This algorithm is highly efficient, executing with a constant time complexity and space complexity since it only utilizes direct arithmetic transformations and a fixed number of variables.